TUYỂN TẬP ĐỀ THI

HỌC SINH GIỎI TIN HỌC THPT

NĂM HỌC 2025 – 2026

HƯỚNG DẪN GIẢI CHI TIẾT

24 đề thi - 96 bài toán

Chương trình C++ đã biên dịch và kiểm chứng

Lê Thành Nam

Lời nói đầu

Quyển sách này tập hợp lời giải chi tiết cho hai mươi bốn đề thi chọn học sinh giỏi Tin học cấp tỉnh bậc THPT năm học 2025 – 2026, gồm tổng cộng 96 bài toán trải từ những bài khởi động vài dòng cho tới những bài đòi hỏi kỹ thuật của kỳ thi quốc gia.

Điều làm nên khác biệt của tài liệu không nằm ở việc đưa ra đáp án. Mỗi bài được dẫn dắt theo đúng con đường mà một thí sinh thật sự đi qua trong phòng thi: đọc đề và phát hiện những chi tiết dễ bỏ sót; nghĩ ra cách làm ngây thơ nhất và tự tay chạy nó trên dữ liệu mẫu; nhận ra chính xác chỗ nó bế tắc; rồi từ chỗ bế tắc ấy mà tìm ra hướng tối ưu, kèm lập luận vì sao hướng mới là đúng.

Toàn bộ chương trình trong sách đều đã được biên dịch bằng g++ với tuỳ chọn -O2, chạy đúng mọi dữ liệu mẫu của đề, đối chiếu chéo với bản ngây thơ trên hàng trăm bộ test ngẫu nhiên và đo thời gian thực tế ở giới hạn lớn nhất. Ở những chỗ thuật toán còn giới hạn thật sự, hoặc đề bài có sai sót, tài liệu ghi chú thẳng thắn thay vì che giấu - vì đó chính là những chỗ học sinh học được nhiều nhất.

Mã nguồn được trình bày theo chuẩn định dạng của sách in: mỗi câu lệnh một dòng, thụt lề bốn dấu cách, dấu ngoặc nhọn đặt đúng quy ước, độ rộng dòng không vượt quá khổ trang. Học sinh có thể chép lại nguyên văn và biên dịch được ngay.

> **CÁCH ĐỌC MỘT BÀI TRONG SÁCH**
> ■ Tóm tắt - phát biểu lại bài toán bằng ngôn ngữ ngắn gọn nhất.
> ■ Đọc kỹ đề - những chi tiết, trường hợp biên và cái bẫy mà đề cài sẵn.
> ■ Ý tưởng ngây thơ - cách nghĩ đầu tiên, kèm ví dụ chạy tay, các bước và độ phức tạp.
> ■ Bế tắc - vì sao cách ngây thơ không đủ, và manh mối dẫn tới lời giải tốt hơn.
> ■ Ý tưởng tối ưu - lời giải chính, kèm phần chứng minh tính đúng đắn.
> ■ Mở rộng - khuôn kỹ thuật dùng sau bài toán và những bài khác dùng chung khuôn đó.
> ■ Nhận xét - kinh nghiệm phòng thi và kết quả kiểm chứng bằng máy.

Lê Thanh Nam

Mục lục

#	Đơn vị ra đề	Số bài	Tên các bài toán
01	Bắc Ninh	4	Độ lệch • Xóa số • Mật khẩu • Dãy con tăng
02	TP. Cần Thơ	5	Khu vườn • Dãy đèn • Dãy số • Vòng tròn • Mạng máy tính
03	TP. Hải Phòng	5	Xuân đảo ngược • Số nguyên đặc biệt • Tìm cặp số • Xóa xâu • Tìm kiếm
04	Hà Tĩnh - Khối 11	3	Số đẹp • Dịch vụ • Lựa chọn
05	Hà Tĩnh - Khối 12	3	Tìm Số • Số cặp nhảy • Hành trình cứu trợ
06	Thành phố Huế	4	Trò chơi với dãy số • Bảng số • Hàng rào • Trồng cây
07	Nghệ An	3	Mã hàng hóa • Nhiên liệu • Hoa đẹp
08	Quảng Ninh	4	Trễ hạn • Đếm dãy con • Phần thưởng • Phân tích cây
09	Quảng Trị	4	Trò chơi điện tử • Thẻ bài • Phần tử thiếu • Bước nhảy không gian
10	Thái Nguyên	4	Hội Xuân • Phân công công việc • Tối ưu hàm • Hệ thống điện
11	An Giang	3	Biến đổi • Số đẹp • Dãy con
12	Đắk Lắk	5	Phương trình • Mật khẩu • Dãy số • Trò chơi • Mạng máy tính
13	Đồng Nai - Bảng A	4	Phát quà • Hàng rào • Cảm biến • Phần tiêu
14	Đồng Nai - Bảng B	5	Số may mắn • Hội thao • Thiết bị bay • Số nguyên tố • Robot
15	Đồng Tháp	4	Số đặc biệt • Bộ ba số nguyên • Kế hoạch xây dựng • Hội thảo
16	Gia Lai	4	Đếm dãy • Xâu đối xứng • Chọn quà • Tải ứng dụng
17	Hưng Yên	4	Chia dư k • Lệch nhỏ nhất • Số lớn nhất • Xe cấp cứu
18	Khánh Hoà	4	Tổng phân số • Dãy tăng liên tiếp • Chia phần • Mảnh đất trồng cây
19	Lâm Đồng	4	Xâu con • Cạm bẫy • Mật mã • Du lịch
20	Lào Cai	5	Ước số • Tìm chuỗi kí tự • Chọn số • Trung vị của dãy số • Gặp gỡ
21	Phú Thọ	4	Dãy số • Đếm số nguyên tố • Xâu con dài nhất • Ăn vặt
22	Quảng Ngãi	4	Bài 1 • Bài 2 • Bài 3 • Bài 4
23	Tuyên Quang	4	Cặp số chẵn • Tổng ước • Tương phản • Cứu trợ
24	Vĩnh Long	3	Xếp thẻ • Chênh lệch • Nhận diện

Lê Thành Nam

01. BẮC NINH

Kỳ thi chọn học sinh giỏi cấp tỉnh THPT - Môn Tin học

*Chú ý: Dấu * được thay thế bởi py hoặc cpp của ngôn ngữ lập trình được sử dụng tương ứng là Python hoặc C++. Tài liệu này trình bày lời giải bằng C++.*

Tổng quan bài thi

Bài	Tên bài	Điểm
1	Độ lệch	5,0 điểm
2	Xóa số	7,0 điểm
3	Mật khẩu	5,0 điểm
4	Dãy con tăng	3,0 điểm

Phần A. Đề bài

Bài 1. Độ lệch *(5,0 điểm)*

Anna là một học sinh yêu thích toán học, đặc biệt là các bài toán liên quan đến dãy số và phần dư của phép chia hai số nguyên. Theo quan điểm của Anna thì độ lệch của hai số nguyên x và y là phần dư của phép chia số lớn cho số bé.

Ví dụ: Với x = 7; y = 4 thì độ lệch của hai số x và y là 3.

Độ lệch của một dãy số nguyên là độ lệch lớn nhất của hai phần tử bất kì trong dãy đó.

Ví dụ: Với dãy số A = (2, 9, 5) thì độ lệch của dãy số A là 4.

Yêu cầu: Cho dãy số A gồm N phần tử nguyên không âm $A_1, A_2, \ldots, A_N$. Tìm độ lệch của dãy số A.

Dữ liệu vào: Đọc từ tệp văn bản DOLECH.INP gồm hai dòng:

- Dòng 1: Ghi số nguyên N là số lượng phần tử của dãy số A ($1 < N \le 2 \times 10^5$);
- Dòng 2: Ghi N số nguyên $A_1, A_2, \ldots, A_N$ ($0 \le A_i \le 10^6$).

Kết quả: Ghi ra tệp văn bản DOLECH.OUT một số nguyên là kết quả của bài toán.

Dữ liệu thử

DOLECH.INP	DOLECH.OUT	Giải thích
3 4 2 7	3	4 chia 2 dư 0 → độ lệch của 4 và 2 là 0; 7 chia 2 dư 1 → độ lệch của 7 và 2 là 1; 7 chia 4 dư 3 → độ lệch của 7 và 4 là 3. → Kết quả cần tìm là 3

Ràng buộc

- Subtask 1: Có 80% số test ứng với 80% số điểm có $N \le 5 \times 10^3$;
- Subtask 2: Có 20% số test ứng với 20% số điểm có $N \le 2 \times 10^5$.

Bài 2. Xóa số *(7,0 điểm)*

Cho dãy A gồm N phần tử nguyên $A_1, A_2, \ldots, A_N$ và số nguyên dương K. Khi thực hiện thao tác xóa K phần tử bất kì khỏi dãy A ta thu được dãy B gồm các phần tử còn lại của dãy A. Gọi M là hiệu lớn nhất giữa hai số bất kỳ trong dãy B, m là hiệu không âm nhỏ nhất giữa hai số bất kỳ trong dãy B.

Lê Thành Nam

Yêu cầu: Tìm giá trị nhỏ nhất của |M| + |m|.

Dữ liệu vào: Đọc từ tệp văn bản XOASO.INP gồm hai dòng:

- Dòng 1: Ghi hai số nguyên dương N và K ($3 \le N \le 10^6$, $1 \le K \le N - 2$);
- Dòng 2: Ghi N số nguyên $A_1, A_2, \ldots, A_N$ ($|A_i| \le 5 \times 10^6$; i = 1, 2, …, N).

Kết quả: Ghi ra tệp văn bản XOASO.OUT một giá trị duy nhất là kết quả của bài toán.

Dữ liệu thử

XOASO.INP	XOASO.OUT	Giải thích
5 2 -3 -2 3 8 6	7	Có thể xóa hai số −3, −2 → dãy B = (3, 8, 6) → M = 5; m = 2 → kết quả bằng 5 + 2 = 7
6 2 -5 8 10 1 13 -1	13	Có thể xóa hai số −5, 13 → dãy B = (8, 10, 1, −1) → M = 11; m = 2 → kết quả bằng 11 + 2 = 13

Ràng buộc

- Subtask 1: Có 30% số test ứng với 30% số điểm có $N \le 20$;
- Subtask 2: Có 40% số test ứng với 40% số điểm có $N \le 10^3$;
- Subtask 3: Có 30% số test ứng với 30% số điểm có $N \le 10^6$.

Bài 3. Mật khẩu *(5,0 điểm)*

Điệp viên Smith có gói tin quan trọng cần bảo mật nên anh đặt mật khẩu. Smith ghi vào sổ dãy A có N số nguyên $A_1, A_2, A_3, \ldots, A_N$ ($1 \le A_i \le 10^6$). Dãy A có ít nhất M giá trị khác nhau. Mật khẩu của Smith là một đoạn con liên tiếp $A_L, A_{L+1}, \ldots, A_R$ ($1 \le L \le R \le N$) ngắn nhất chứa đủ M giá trị khác nhau. Khi đó, giá trị R − L + 1 là độ dài của mật khẩu.

Do Smith được giao quá nhiều nhiệm vụ quan trọng và đột xuất nên tạm thời anh ấy bị quên mất mật khẩu. Smith muốn dò lại mật khẩu từ dãy số A nhưng để việc tìm kiếm thuận lợi anh cần tìm chính xác độ dài của mật khẩu đó.

Yêu cầu: Bạn hãy giúp Smith tìm độ dài của mật khẩu.

Dữ liệu vào: Đọc từ tệp văn bản MATKHAU.INP gồm hai dòng:

- Dòng 1: Ghi 2 số nguyên dương N, M ($N \le 5 \times 10^6$, $M \le N$);
- Dòng 2: Ghi N số nguyên dương $A_1, A_2, A_3, \ldots, A_N$ ($1 \le A_i \le 10^6$).

Kết quả: Ghi ra tệp văn bản MATKHAU.OUT một số nguyên duy nhất là kết quả của bài toán.

Dữ liệu thử

MATKHAU.INP	MATKHAU.OUT
10 4 5 2 1 1 3 2 1 3 2 1	5

Đoạn con (5, 2, 1, 1, 3) có đủ 4 giá trị khác nhau {5, 2, 1, 3} và không có đoạn nào ngắn hơn thoả mãn.

Ràng buộc

- Subtask 1: Có 60% số test ứng với 60% số điểm có $1 \le N \le 10^2$;
- Subtask 2: Có 40% số test ứng với 40% số điểm có $10^2 < N \le 5 \times 10^6$.

Bài 4. Dãy con tăng *(3,0 điểm)*

Lê Thanh Nam

Tom rất yêu thích việc liệt kê các dãy con (đặc biệt là các dãy con tăng) của một dãy số nguyên. Ta gọi A′ là một dãy con tăng của dãy A nếu A′ thu được bằng cách bỏ đi một số phần tử của A, giữ nguyên thứ tự các phần tử còn lại và các phần tử trong A′ được sắp xếp theo thứ tự tăng nghiêm ngặt.

Ví dụ: Cho dãy A = [6, 1, 9, 5, 8, 5]. Các dãy [1, 5, 8] và [1, 8] là dãy con tăng của A nhưng [1, 5, 5] và [6, 1, 5] thì không phải.

Với niềm đam mê tìm hiểu các con số, Tom định nghĩa độ khó của một số nguyên là tích các chữ số của nó.

Ví dụ: Hai số 1234 và 813 đều có độ khó bằng 24, còn số 8 có độ khó bằng chính nó.

Tom viết ra một dãy số A nguyên ngẫu nhiên, không có phần tử âm và đố Been tìm ra độ khó lớn nhất của dãy con tăng nghiêm ngặt trong dãy A. Biết rằng độ khó của một dãy số được quy ước là tổng độ khó của tất cả các phần tử trong dãy đó.

Yêu cầu: Cho dãy A gồm N số nguyên không âm $A_1, A_2, \ldots, A_N$. Tìm độ khó lớn nhất của dãy con tăng trong dãy A.

Dữ liệu vào: Đọc từ tệp văn bản DAYCON.INP gồm hai dòng:

- Dòng 1: Ghi số nguyên dương N ($1 \le N \le 2 \times 10^5$);
- Dòng 2: Ghi N số nguyên không âm $A_1, A_2, A_3, \ldots, A_N$ ($0 \le A_i \le 10^9$; i = 1, 2, …, N).

Kết quả: Ghi ra tệp văn bản DAYCON.OUT một số nguyên là kết quả cần tìm.

Dữ liệu thử

DAYCON.INP	DAYCON.OUT	Giải thích
5 5 101 54 123 56	55	Dãy [5, 54, 56] có độ khó lớn nhất trong số các dãy con tăng. Độ khó của nó là 5 + 20 + 30 = 55

Ràng buộc

- Subtask 1: Có 20% số test ứng với 20% số điểm có N ≤ 20;
- Subtask 2: Có 20% số test ứng với 20% số điểm có N ≤ 2000;
- Subtask 3: Có 60% số test ứng với 60% số điểm có $N \le 2 \times 10^5$.

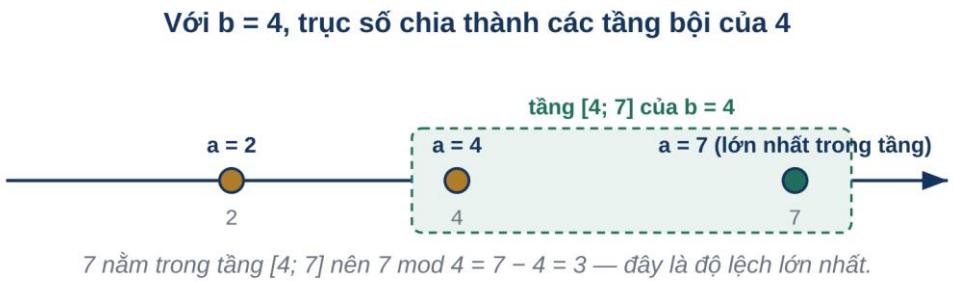
Lê Thành Nam

Phần B. Hướng dẫn giải

BÀI 1

Độ lệch

Cho dãy N số nguyên không âm ($N \le 2\times10^5$, giá trị $\le 10^6$). Với mỗi cặp phần tử, lấy số lớn chia số bé và giữ lại phần dư. Cần tìm phần dư lớn nhất trên toàn bộ các cặp.



Hình 1. Cố định số bé b, phần dư lớn nhất đạt tại phần tử lớn nhất của mỗi tầng.

■ Đọc kỹ đề

Có một chi tiết dễ bỏ sót: đề cho phép $A_i = 0$. Nếu số bé bằng 0 thì phép chia không thực hiện được, nên cặp đó phải bỏ qua. Ngoài ra hai phần tử bằng nhau luôn cho phần dư 0, nên không bao giờ làm hỏng kết quả.

■ Ý tưởng ngây thơ

Bài toán phát biểu trên "mọi cặp", nên cách nghĩ đầu tiên hoàn toàn tự nhiên: liệt kê tất cả các cặp (i, j) với i < j, xác định đâu là số lớn, đâu là số bé, tính phần dư rồi giữ lại giá trị lớn nhất.

Đây là bản mô phỏng nguyên văn định nghĩa trong đề, nên gần như không thể sai; nó cũng chính là "thước đo" để đối chiếu với bản tối ưu về sau.

> **VÍ DỤ CHẠY TAY**
> Với A = (4, 2, 7): cặp (4, 2) cho 4 mod 2 = 0; cặp (4, 7) cho 7 mod 4 = 3; cặp (2, 7) cho 7 mod 2 = 1. Giá trị lớn nhất là 3.

Thuật toán

Bước 1. Đọc N và dãy A.

Bước 2. Đặt kết quả ans = 0.

Bước 3. Với mọi cặp chỉ số i < j: đặt x = max(A_i, A_j), y = min(A_i, A_j); nếu y > 0 thì cập nhật ans = max(ans, x mod y).

Bước 4. In ans.

Độ phức tạp. $O(N^2)$ thời gian, O(N) bộ nhớ. Với $N \le 5\times10^3$ (Subtask 1) chỉ khoảng $1{,}2\times10^7$ phép toán - chạy tốt và đủ lấy 80% số điểm.

▸ ***Chương trình - bản ngây thơ***

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios_base::sync_with_stdio(false);
```

Lê Thành Nam

```
    cin.tie(nullptr);
    if (fopen("DOLECH.INP", "r")) {
        freopen("DOLECH.INP", "r", stdin);
        freopen("DOLECH.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    vector<int> a(n);
    for (auto& x : a)
        cin >> x;
    int ans = 0;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) {
            int x = max(a[i], a[j]), y = min(a[i], a[j]);
            if (y)
                ans = max(ans, x % y);
        }
    cout << ans << "\n";
}
```

■ Bế tắc ở đâu?

VÌ SAO CHƯA TỐI ƯU

Với $N = 2\times10^5$, số cặp là khoảng 2×10^{10}. Dù mỗi cặp chỉ tốn vài phép tính, chương trình vẫn cần hàng phút - vượt xa giới hạn thời gian.

Điều đáng chú ý là số lượng cặp lớn nhưng số lượng giá trị khác nhau thì không: mọi A_i đều nằm trong đoạn $[0; 10^6]$. Nghĩa là trong 2×10^5 phần tử chắc chắn có rất nhiều giá trị lặp lại, và các cặp có cùng bộ giá trị thì cho cùng một phần dư. Vậy nên duyệt theo giá trị, chứ không duyệt theo chỉ số.

■ Ý tưởng tối ưu

Ta cố định số bé b và hỏi: với b cố định, phần dư a mod b lớn nhất là bao nhiêu?

Trục số được b chia thành các "tầng": [b; 2b−1], [2b; 3b−1], [3b; 4b−1], … Nếu a nằm trong tầng thứ k, tức $k\cdot b \le a \le (k+1)\cdot b - 1$, thì $a \bmod b = a - k\cdot b$. Trong cùng một tầng, phần dư tăng theo a, nên chỉ cần lấy a lớn nhất có mặt trong tầng đó.

Như vậy với mỗi giá trị b có mặt trong dãy, ta duyệt lần lượt các tầng và mỗi tầng chỉ cần trả lời một câu hỏi: "giá trị lớn nhất có mặt trong dãy mà không vượt quá một mốc cho trước là bao nhiêu?". Câu hỏi này được trả lời trong O(1) nếu dựng trước mảng mx[v] = giá trị lớn nhất xuất hiện trong dãy và ≤ v (một lượt quét tiền tố trên đoạn giá trị).

VÌ SAO LỜI GIẢI ĐÚNG

Với b cố định, mỗi $a \ge b$ thuộc đúng một tầng $k = \lfloor a/b \rfloor$ và $a \bmod b = a - k\cdot b$ là hàm tăng theo a trong tầng đó. Do đó cực đại của a mod b theo a chính là cực đại theo từng tầng, mà cực đại trong một tầng đạt tại phần tử lớn nhất của dãy nằm trong tầng - đúng bằng mx[min(maxA, (k+1)·b − 1)] nếu giá trị này còn ≥ k·b. Duyệt hết các tầng của mọi b nghĩa là đã xét hết mọi cặp (số bé, số lớn) có thể.

VÍ DỤ CHẠY TAY

Với A = (4, 2, 7), maxA = 7, mảng mx là 0,0,2,2,4,4,4,7 (theo v = 0..7). Với b = 2: tầng [2;3] cho mx[3] = 2 → dư 0; tầng [4;5] cho mx[5] = 4 → dư 0; tầng [6;7] cho mx[7] = 7 → dư 1. Với b = 4: tầng [4;7] cho mx[7] = 7 → dư 3. Với b = 7: tầng [7;7] cho dư 0. Kết quả 3.

Thuật toán

Bước 1. Đọc dãy, đánh dấu has[v] = true với mọi giá trị v xuất hiện, ghi nhận maxA.

Bước 2. Quét v từ 0 đến maxA, dựng mx[v] = giá trị lớn nhất đã gặp cho tới v (nếu chưa có thì −1).

Lê Thành Nam

Bước 3. Với mỗi b từ 1 đến maxA mà has[b] = true: cho mốc lo chạy theo bội của b (lo = b, 2b, 3b, …, ≤ maxA), đặt up = min(maxA, lo + b − 1); nếu mx[up] ≥ lo thì cập nhật ans = max(ans, mx[up] − lo).

Bước 4. In ans. Giá trị b = 0 bị loại ngay từ vòng lặp nên không có phép chia cho 0.

Độ phức tạp. Số lần lặp là Σ_b (maxA / b) ≤ maxA · (1 + 1/2 + 1/3 + …) ≈ maxA·ln(maxA), tức khoảng $1{,}4\times10^7$ với maxA = 10^6 - đây là kỹ thuật "duyệt theo bội số" quen thuộc của sàng điều hoà. Bộ nhớ O(maxA). Thực đo: 0,03 giây với N = 2×10^5 giá trị ngẫu nhiên trong [0; 10^6].

▸ ***Chương trình - bản tối ưu***

```cpp
#include <bits/stdc++.h>
using namespace std;
int mx[1000005];
bool has_[1000005];
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("DOLECH.INP", "r")) {
        freopen("DOLECH.INP", "r", stdin);
        freopen("DOLECH.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    int hi = 0;
    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;
        has_[x] = 1;
        if (x > hi)
            hi = x;
    }
    int cur = -1;
    for (int v = 0; v <= hi; v++) {
        if (has_[v])
            cur = v;
        mx[v] = cur;
    }
    int ans = 0;
    for (int b = 1; b <= hi; b++) {
        if (!has_[b])
            continue;
        for (int lo = b; lo <= hi; lo += b) {
            int up = min(hi, lo + b - 1);
            if (mx[up] >= lo && mx[up] - lo > ans)
                ans = mx[up] - lo;
        }
    }
    cout << ans << "\n";
}
```

Lê Thành Nam

■ Mở rộng

MỞ RỘNG KỸ THUẬT

Kỹ thuật "duyệt theo bội số" (tổng điều hoà) dùng lại cho hàng loạt bài: tổng ⌊n/i⌋, đếm cặp có cùng thương, sàng số ước, sàng hàm Euler. Hễ thấy miền giá trị nhỏ hơn nhiều so với số cặp cần xét thì hãy đổi trục duyệt từ chỉ số sang giá trị.

■ Nhận xét

Ý tưởng cốt lõi là chuyển từ "duyệt theo chỉ số" sang "duyệt theo giá trị". Đây là bước chuyển rất hay gặp khi miền giá trị nhỏ hơn nhiều so với số cặp cần xét.

Kỹ thuật chia trục số thành các tầng bội của b và chỉ lấy phần tử lớn nhất mỗi tầng còn dùng lại trong nhiều bài khác: tìm max của a mod b, đếm số cặp có thương bằng k, tổng ⌊n/i⌋…

Bẫy cài trong đề: giá trị 0 (không được làm số chia) và các giá trị trùng nhau (chỉ cần xét một lần nhờ mảng has).

Kiểm chứng. Bản tối ưu và bản ngây thơ cho kết quả trùng khớp trên 400 bộ test ngẫu nhiên ($N \le 60$, giá trị trong [0; 1000], có cả số 0 và số trùng nhau); hai ví dụ trong đề đều đúng; thời gian chạy 0,03 giây ở giới hạn lớn nhất.

BÀI 2

Xóa số

Giữ lại đúng $L = N - K$ phần tử ($L \ge 2$) sao cho tổng của "khoảng cách lớn nhất giữa hai phần tử còn lại" và "khoảng cách nhỏ nhất giữa hai phần tử còn lại" là nhỏ nhất.



Hình 1. Trượt cửa sổ độ dài L, lấy biên độ cộng hiệu liền kề nhỏ nhất trong cửa sổ.

■ Đọc kỹ đề

Hai đại lượng M và m đều là hiệu không âm nên dấu giá trị tuyệt đối chỉ mang tính hình thức: sau khi sắp xếp dãy B, M là phần tử cuối trừ phần tử đầu, còn m là hiệu nhỏ nhất giữa hai phần tử đứng liền nhau.

■ Ý tưởng ngây thơ

Cách trực tiếp nhất: thử tất cả các cách chọn L phần tử giữ lại. Với mỗi cách chọn, sắp xếp lại, lấy phần tử lớn nhất trừ nhỏ nhất được M, quét các cặp liền kề lấy hiệu nhỏ nhất được m, rồi cập nhật đáp số.

Bản này chậm nhưng cực kỳ có ích: nó là bằng chứng để kiểm tra nhận định "tập giữ lại tối ưu luôn là một đoạn liên tiếp sau khi sắp xếp" ở phần sau.

> **VÍ DỤ CHẠY TAY**
> Với $N = 5$, $K = 2$, $A = (-3, -2, 3, 8, 6)$: trong 10 cách giữ lại 3 phần tử, cách giữ (3, 8, 6) cho $M = 5$, $m = 2$, tổng 7; cách giữ (−3, −2, 3) cho $M = 6$, $m = 1$, tổng 7; các cách còn lại đều lớn hơn. Đáp số 7.

Thuật toán

Bước 1. Đọc N, K, dãy A; đặt $L = N - K$.

Bước 2. Duyệt mọi tập con gồm L chỉ số (dùng mặt nạ nhị phân hoặc next_permutation trên mảng 0/1).

Bước 3. Với mỗi tập con: sắp xếp các giá trị được giữ, tính M = giá trị cuối − giá trị đầu và m = min hiệu hai phần tử liền nhau, cập nhật kết quả nhỏ nhất của M + m.

Bước 4. In kết quả.

Độ phức tạp. $O(C(N, L) \cdot L \log L)$. Chỉ chạy được với $N \le 20$ (Subtask 1), tức khoảng 30% số điểm.

▸ ***Chương trình - bản ngây thơ***

```
#include <bits/stdc++.h>
```

Lê Thành Nam

```cpp
using namespace std;
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("XOASO.INP", "r")) {
        freopen("XOASO.INP", "r", stdin);
        freopen("XOASO.OUT", "w", stdout);
    }
    int n, k;
    cin >> n >> k;
    vector<long long> a(n);
    for (auto& x : a)
        cin >> x;
    int L = n - k;
    long long best = LLONG_MAX;
    vector<int> id(n);
    for (int i = 0; i < n; i++)
        id[i] = i;
    vector<int> cmb(n, 0);
    for (int i = 0; i < L; i++)
        cmb[n - 1 - i] = 1;
    do {
        vector<long long> b;
        for (int i = 0; i < n; i++)
            if (cmb[i])
                b.push_back(a[i]);
        sort(b.begin(), b.end());
        long long M = b.back() - b[0], m = LLONG_MAX;
        for (size_t i = 0; i + 1 < b.size(); i++)
            m = min(m, b[i + 1] - b[i]);
        best = min(best, M + m);
    } while (next_permutation(cmb.begin(), cmb.end()));
    cout << best << "\n";
}
```

■ Bế tắc ở đâu?

> **VÌ SAO CHƯA TỐI ƯU**
>
> Số tập con tăng theo hàm mũ: với N = 30 đã có hơn 10^8 khả năng, còn N = 10^6 thì hoàn toàn nằm ngoài tầm với.
>
> Muốn tiến xa hơn phải trả lời được câu hỏi: trong vô số tập con, tập tối ưu có hình dạng đặc biệt nào không? Trực giác gợi ý rằng cả hai đại lượng cần tối thiểu đều "thích" các phần tử nằm sát nhau trên trục số.

■ Ý tưởng tối ưu

Sắp xếp dãy A tăng dần. Khẳng định then chốt: tồn tại phương án tối ưu trong đó L phần tử giữ lại là L phần tử liên tiếp của dãy đã sắp xếp.

Khi đã có khẳng định này, bài toán rút gọn thành: trượt một cửa sổ độ dài L trên dãy đã sắp; với cửa sổ bắt đầu tại vị trí s, ta có M = a[s+L−1] − a[s], còn m là giá trị nhỏ nhất của mảng hiệu liền kề gap[i] = a[i+1] − a[i] trên đoạn chỉ số [s; s+L−2]. Lấy min của M + m trên mọi s.

Việc lấy min trên một cửa sổ trượt độ dài cố định được xử lý bằng hàng đợi đơn điệu (deque), tổng chi phí O(N).

> **VÌ SAO LỜI GIẢI ĐÚNG**
>
> Giả sử tập tối ưu S có các chỉ số (sau khi sắp xếp) là $i_1 < i_2 < \ldots < i_L$, đạt giá trị m = d tại một cặp liền nhau i_j, i_{j+1}. Hiệu $a[i_{j+1}] - a[i_j] = d$ là tổng của các hiệu liền kề của dãy gốc nằm giữa hai vị trí đó, nên tồn tại một cặp liền kề (p, p+1) với gap[p] ≤ d nằm trong đoạn $[i_1; i_L]$. Vì $i_L - i_1 + 1 \geq L$ nên chọn được cửa sổ liên tiếp W độ dài L nằm trong $[i_1; i_L]$ và chứa cặp (p, p+1). Khi đó m(W) ≤ gap[p] ≤ d và $M(W) \leq a[i_L] - a[i_1] = M(S)$. Vậy W không tệ hơn S. Khẳng định này còn được kiểm tra lại

Lê Thành Nam

bằng máy: đối chiếu với bản duyệt mọi tập con trên 500 bộ test ngẫu nhiên, kết quả trùng khớp hoàn toàn.

VÍ DỤ CHẠY TAY

Ví dụ 2: A sắp xếp thành (−5, −1, 1, 8, 10, 13), gap = (4, 2, 7, 2, 3), L = 4. Cửa sổ (−5, −1, 1, 8): M = 13, m = min(4, 2, 7) = 2 → 15. Cửa sổ (−1, 1, 8, 10): M = 11, m = min(2, 7, 2) = 2 → 13. Cửa sổ (1, 8, 10, 13): M = 12, m = min(7, 2, 3) = 2 → 14. Nhỏ nhất là 13.

Thuật toán

Bước 1. Đọc dữ liệu bằng hàm đọc nhanh (fread + tự tách số) vì N có thể tới 10^6.

Bước 2. Sắp xếp dãy A tăng dần; đặt L = N − K, dựng mảng gap[i] = a[i+1] − a[i] gồm N − 1 phần tử.

Bước 3. Cho i chạy trên mảng gap, duy trì deque các chỉ số có gap tăng dần: trước khi đẩy i vào, loại khỏi cuối deque mọi chỉ số có gap lớn hơn hoặc bằng gap[i]; loại khỏi đầu deque chỉ số đã rời khỏi cửa sổ độ rộng L − 1.

Bước 4. Khi cửa sổ đã đủ L − 1 phần tử (i ≥ L − 2), gọi s = i − L + 2 là vị trí đầu cửa sổ: cập nhật kết quả với (a[s+L−1] − a[s]) + gap[đầu deque].

Bước 5. In kết quả nhỏ nhất tìm được.

Độ phức tạp. O(N log N) cho phép sắp xếp và O(N) cho cửa sổ trượt; bộ nhớ O(N). Thực đo: 0,12 giây với N = 10^6, K = N/2, giá trị ngẫu nhiên trong [−5×10^6; 5×10^6].

▸ ***Chương trình - bản tối ưu***

```
#include <bits/stdc++.h>
using namespace std;
static char buf[1 << 16];
int rdpos = 0, rdlen = 0;
inline int gc() {
    if (rdpos == rdlen) {
        rdlen = (int)fread(buf, 1, sizeof(buf), stdin);
        rdpos = 0;
        if (rdlen <= 0)
            return -1;
    }
    return buf[rdpos++];
}
inline long long rdint() {
    int c = gc();
    while (c != '-' && (c < '0' || c > '9'))
        c = gc();
    int s = 1;
    if (c == '-') {
        s = -1;
        c = gc();
    }
    long long x = 0;
    while (c >= '0' && c <= '9') {
        x = x * 10 + (c - '0');
        c = gc();
    }
    return x * s;
}
int a[1000006], dq[1000006];
int main() {
    if (fopen("XOASO.INP", "r")) {
        freopen("XOASO.INP", "r", stdin);
        freopen("XOASO.OUT", "w", stdout);
    }
    int n = (int)rdint(), k = (int)rdint();
    for (int i = 0; i < n; i++)
        a[i] = (int)rdint();
```

Lê Thành Nam

```
    sort(a, a + n);
    int L = n - k;
    long long best = LLONG_MAX;
    if (L == 1) {
        cout << 0 << "\n";
        return 0;
    }
    int g = n - 1;
    static int gap[1000006];
    for (int i = 0; i < g; i++)
        gap[i] = a[i + 1] - a[i];
    int h = 0, t = 0;
    int w = L - 1;
    for (int i = 0; i < g; i++) {
        while (t > h && gap[dq[t - 1]] >= gap[i])
            t--;
        dq[t++] = i;
        while (dq[h] <= i - w)
            h++;
        if (i >= w - 1) {
            int s = i - w + 1;
            long long M = (long long)a[s + L - 1] - a[s];
            long long m = gap[dq[h]];
            best = min(best, M + m);
        }
    }
    cout << best << "\n";
}
```

■ Mở rộng

MỞ RỘNG KỸ THUẬT

Bổ đề "tập tối ưu là một đoạn liên tiếp sau khi sắp xếp" xuất hiện trong rất nhiều bài chọn k phần tử tối thiểu hoá biên độ. Khi đã rút về đoạn, mọi đại lượng trên đoạn (min, max, min hiệu liền kề) đều duy trì được bằng cửa sổ trượt.

■ Nhận xét

Bài này là ví dụ đẹp về "đưa bài toán tổ hợp về bài toán trên đoạn liên tiếp": ngay khi chứng minh được tập tối ưu là một đoạn, không gian tìm kiếm rơi từ hàm mũ xuống tuyến tính.

Chú ý kiểu dữ liệu: $|A_i| \le 5\times10^6$ nên M có thể tới 10^7, vẫn nằm trong int, nhưng dùng long long cho biến kết quả là thói quen an toàn.

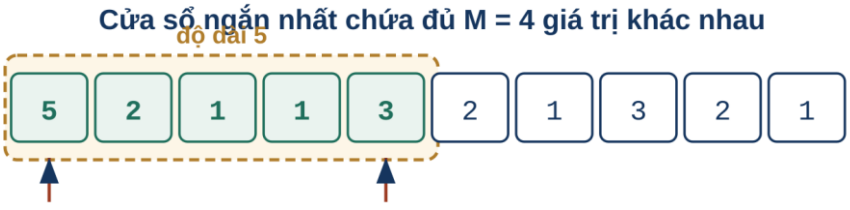
Với $N = 10^6$, cin thông thường (dù đã tắt đồng bộ) vẫn khá nặng; hàm đọc nhanh bằng fread giúp phần nhập liệu gần như không đáng kể trong tổng thời gian.

Kiểm chứng. Đối chiếu với bản duyệt mọi tập con trên 500 bộ test ngẫu nhiên ($N \le 10$, giá trị trong $[-15; 15]$, K ngẫu nhiên) - trùng khớp tuyệt đối; hai ví dụ trong đề đều đúng; 0,12 giây ở giới hạn $N = 10^6$.

BÀI 3

Mật khẩu

Tìm độ dài đoạn con liên tiếp ngắn nhất của dãy A chứa ít nhất M giá trị đôi một khác nhau.



Lê Thành Nam

Hình 1. Hai con trỏ L, R chỉ tiến về một phía; mỗi chỉ số vào và ra cửa sổ đúng một lần.

■ Đọc kỹ đề

Giới hạn $N \le 5\times10^6$ là điểm đáng chú ý nhất của bài: thuật toán phải tuyến tính và phần đọc dữ liệu cũng phải được viết cẩn thận, nếu không riêng khâu nhập liệu đã hết thời gian.

■ Ý tưởng ngây thơ

Duyệt mọi vị trí đầu L. Với mỗi L, mở rộng dần R và dùng một tập hợp (set) để đếm số giá trị khác nhau đã gặp; ngay khi đủ M giá trị thì dừng, ghi nhận độ dài $R - L + 1$ rồi chuyển sang L tiếp theo.

Việc dừng ngay khi đủ là hợp lý vì kéo dài thêm chỉ làm đoạn dài hơn.

> **VÍ DỤ CHẠY TAY**
> Với A = (5, 2, 1, 1, 3, 2, 1, 3, 2, 1), M = 4: bắt đầu L = 1, mở rộng tới R = 5 thì tập giá trị là {5, 2, 1, 3} đủ 4 phần tử, độ dài 5. Với L = 2, phải mở rộng tới hết dãy cũng chỉ có {2, 1, 3} nên không có đoạn nào. Kết quả 5.

Thuật toán

Bước 1. Đọc N, M và dãy A.

Bước 2. Với mỗi L từ 1 đến N: khởi tạo một set rỗng, cho R chạy từ L đến N, thêm A_R vào set; nếu kích thước set đạt M thì cập nhật kết quả với $R - L + 1$ và thoát vòng lặp R.

Bước 3. In kết quả nhỏ nhất.

Độ phức tạp. $O(N^2 \log N)$ trong trường hợp xấu. Chạy tốt với $N \le 10^2$ (Subtask 1), đủ 60% số điểm.

▸ ***Chương trình - bản ngây thơ***

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("MATKHAU.INP", "r")) {
        freopen("MATKHAU.INP", "r", stdin);
        freopen("MATKHAU.OUT", "w", stdout);
    }
    int n, m;
    cin >> n >> m;
    vector<int> a(n);
    for (auto& x : a)
        cin >> x;
    int best = INT_MAX;
    for (int l = 0; l < n; l++) {
        set<int> s;
        for (int r = l; r < n; r++) {
            s.insert(a[r]);
            if ((int)s.size() >= m) {
                best = min(best, r - l + 1);
                break;
            }
        }
    }
    cout << (best == INT_MAX ? -1 : best) << "\n";
}
```

■ Bế tắc ở đâu?

> **VÌ SAO CHƯA TỐI ƯU**
> Với $N = 5\times10^6$, chỉ riêng số cặp (L, R) đã là con số thiên văn. Ngay cả việc khởi tạo lại set cho mỗi L cũng đã quá tốn kém.

Lê Thành Nam